

ONLINE ACTIVITY

Spark Programming with Python: PySpark, SparkContext, and SQLContext

Coverage: Topic 1 – Introduction to PySpark; Topic 2 – Understanding SparkContext; Topic 3 – Exploring SQLContext in Spark

⌚ TOTAL TIME: 120 MINUTES

Activity	Time	Output
1. Online Individual Activity	25 min	Short answers
2. Case Study	30 min	Case analysis
3. Case Scenario	30 min	Problem-solving response
4. Essay	30 min	300–400 word essay
5. Submission/Review	5 min	Final submission
TOTAL	120 min	

ACTIVITY 1 – ONLINE INDIVIDUAL ACTIVITY

“Understand the Spark Workflow” — 25 minutes

Instructions: Answer the following questions based on Topics 1–3.

Part A – Identify

- **What is PySpark?** It is the Python API for Apache Spark, allowing developers to write distributed Spark applications using Python syntax.
- **What is the role of Apache Spark?** It is an open-source, distributed computing engine designed for lightning-fast processing and analytics of large-scale datasets across a cluster of machines.
- **What is the purpose of SparkContext?** It acts as the primary entry point for Spark core functionality. It establishes the connection to the Spark execution environment, allocates resources, and manages the creation of Resilient Distributed Datasets (RDDs).
- **What is the purpose of SQLContext?** It is the entry point for working with structured and semi-structured data in Spark. It allows developers to create DataFrames and execute SQL queries directly on distributed datasets.
- **What is the difference between SparkContext and SQLContext?** SparkContext is used for low-level cluster connection and unstructured data processing (RDDs), whereas SQLContext sits on top of SparkContext and is specifically designed for handling structured data using high-level DataFrames and SQL operations.

Part B – Arrange the Process

Arrange the following components in the correct logical order:

Executors – Python Application – SparkContext – Tasks – Spark Execution Environment

Answer: Python Application → SparkContext → Spark Execution Environment → Executors → Tasks

Part C – Application Structure

Action	Stage
Import PySpark modules	Import
Create the Spark entry point	Initialize
Perform Spark operations	Process
Terminate the application	Stop

Basic application structure: Import → Initialize → Configure → Process → Stop.

ACTIVITY 2 – CASE STUDY

“Student Performance Data Analysis” — 30 minutes

A university has a large student-performance dataset containing Student ID, Student Name, Course, Score, and Year Level. The university wants to analyze the data using PySpark because the data is too large to process efficiently using a simple Python program. The IT team decides to use Apache Spark through PySpark and needs to establish the Spark environment and work with structured student data using SQL.

Questions

- PySpark is highly appropriate for this case because the university's dataset has grown too large for a standard, single-threaded Python script to handle efficiently. By utilizing PySpark, the data processing workload can be distributed across a cluster, enabling parallel processing of the massive student records.

- In this setup, **SparkContext** acts as the central coordinator, connecting the application to the cluster manager to distribute tasks to various nodes. Meanwhile, **SQLContext** allows the IT team to load the structured student data (ID, Name, Course, Score) into DataFrames. Because the requirement is to find students with a Score ≥ 75 , SQLContext enables the execution of familiar SQL queries to filter this structured data rapidly. If the application failed to properly establish its connection to the Spark execution environment via SparkContext, the program would crash or fallback to processing the massive dataset on a single local machine, immediately bringing back the severe performance bottlenecks the team was trying to avoid.

Expected output: 1–2 paragraphs or a short bullet-point analysis.

ACTIVITY 3 – CASE SCENARIO

“The Slow Student Analytics System” — 30 minutes

A college currently uses a Python application to analyze millions of student records. The application loads student data, filters students according to their scores, calculates the average score, and produces a report. However, the application is becoming slow as the amount of data increases. The development team proposes using PySpark, SparkContext, and SQLContext.

Your Task: You are the Data Processing Consultant. Prepare a recommendation explaining how the team should redesign the application.

- **PySpark:** The team should transition to PySpark to break free from the limitations of a single-node Python application. PySpark will allow the application to process millions of student records in parallel, vastly reducing report generation time.
- **SparkContext:** This will serve as the bridge between your Python logic and the distributed cluster. It will take the heavy filtering and calculation jobs and delegate them to multiple worker nodes simultaneously.
- **SQLContext:** Since the student records are highly structured, SQLContext will transform the raw data into DataFrames. This provides a highly optimized execution engine that allows the team to manipulate the data using standard relational database paradigms.
- **Processing:** The SQL query used to filter the students would be: `SELECT * FROM students WHERE score >= 75`
- **Architecture:** Python Application → **SparkContext** → **Spark Execution Environment** → Data Processing

ACTIVITY 4 – INDIVIDUAL ESSAY

30 minutes

Essay Question: “Why are PySpark, SparkContext, and SQLContext important when developing applications for large-scale data processing?”

Write 300–400 words.

- When dealing with massive datasets, traditional single-machine programs often hit severe performance bottlenecks. This is where Apache Spark, a highly efficient distributed computing framework, becomes essential. PySpark is the Python API for Apache Spark, effectively bridging the gap between Python’s highly readable syntax and Spark’s powerful, distributed processing capabilities. Through PySpark, developers can write Python code that executes in parallel across a massive cluster of machines.
- At the heart of this integration is the SparkContext. It acts as the primary gateway, connecting the Python application to the Spark execution environment. SparkContext manages resource allocation, coordinates with the cluster manager, and breaks down complex jobs into smaller tasks that are distributed across worker nodes. However, raw distributed data can be complex to manipulate.
- This is where SQLContext proves invaluable. While standard relational database management systems like MySQL, PostgreSQL, or SQL Server handle structured data efficiently on a smaller scale, SQLContext brings

that same familiar, structured querying paradigm to massive, distributed datasets. It allows developers to execute optimized SQL queries on data scattered across hundreds of machines, making data analysis highly scalable.

- In a real-world application, these components operate in harmony. A developer writes a Python script using PySpark, initializing a SparkContext to connect to the cluster. An SQLContext is then created to load structured data, allowing the application to execute high-level SQL aggregations before SparkContext distributes the tasks for execution.
- For example, consider a robust Web-Based Logistics Document Management System integrated with a Procurement System. If the platform accumulates millions of daily tracking documents and procurement receipts, querying this data via standard Microsoft Access or a single-instance database would cause severe latency. By deploying PySpark, the system uses SparkContext to distribute the workload across a cluster, while SQLContext seamlessly runs complex analytical queries—such as identifying average procurement delivery times across different regions—on the massive dataset in a fraction of the time.

•

ONLINE SUBMISSION

Name: _____

Section: _____

Date: _____

Required Sections: I. Online Individual Activity | II. Case Study | III. Case Scenario | IV. Essay

GRADING RUBRIC – 100 POINTS

Component	Points
Online Activity	20
Case Study Analysis	25
Case Scenario / Problem Solving	25
Essay	30
TOTAL	100

Essay Rubric – 30 points

Criteria	Points
Understanding of PySpark	6
Understanding of SparkContext	6
Understanding of SQLContext	6
Application to real-world situation	6
Organization, clarity, and grammar	6
TOTAL	30

Online Mode: Students may complete the activities individually using the uploaded lecture presentations as their primary reference.